

## Pwd Command

```
rgukt@rgukt-lenovo-e41-55: ~$ pwd
```

```
//home/rgukt
```

## Mkdir Command

```
rgukt@rgukt-lenovo-e41-55: ~$ mkdir ankitha
```

```
rgukt@rgukt-lenovo-e41-55: ~$ ls
```

Cprograms    java    operating-system    HTML

ankitha    css    python

## Rmdir Command

```
rgukt@rgukt-lenovo-e41-55: ~$ rmdir ankitha
```

```
rgukt@rgukt-lenovo-e41-55: ~$ ls
```

Cprograms    java    operating-system

css    python    HTML

## Cd Command

```
rgukt@rgukt-lenovo-e41-55: ~$ cd ankitha
```

```
rgukt@rgukt-lenovo-e41-55: ~$ /ankitha$
```

closing the directory

```
rgukt@rgukt-lenovo-e41-55: ~$ cd ..
```

```
rgukt@rgukt-lenovo-e41-55: ~$
```

## LAB-1

### Linux Commands

1. pwd - present working directory

Description:-

→ It prints the name of the current working directory

Syntax:- pwd

Command:- \$ pwd

2. Mkdir - make directory

→ The mkdir command is used to create the directory (directories), if they do not already exist.

Syntax:- mkdir <directory\_name>

Command:- \$ mkdir operating-system

3. Rmdir - remove directory

→ It is used to remove the directory (directories), if they are empty.

Syntax: rmdir <directory name>

Command: \$ rmdir operating-system

4. Cd - change directory

Description:- change the shell working directory.

Syntax:- cd <directory name>

Command:- \$ cd Desktop

→ change the current directory to DIR. The default DIR is the value of the HOME shell variable.

→ For closing present working directory we use "cd.."

### ls Command

```
rgukt@rgukt -lenovo-e41-55:~$ cd Desktop
rgukt@rgukt -lenovo-e41-55:~/Desktop$ ls
abstract.tex      cprograms        operatingSystem-lab
adc.c             dsp-project      --pycache--
calculator.py      dsp-project.zip  recursion
```

### Cat Command

```
rgukt@rgukt -lenovo-e41-55:~/ankitha$ cat > file1.txt
Hello RGUKT!
//After pressing 'CTRL+D'
rgukt@rgukt -lenovo-e41-55:~/ankitha$ cat file1.txt
Hello RGUKT!
```

### touch Command:-

```
rgukt@rgukt -lenovo-e41-55:~/ankitha$ touch file2.txt
rgukt@rgukt -lenovo-e41-55:~/ankitha$ ls
file1.txt  file2.txt
//For creating multiple files at a time
rgukt@rgukt -lenovo-e41-55:~/ankitha$ touch file1 file2
rgukt@rgukt -lenovo-e41-55:~/ankitha$ ls
file1 file1.txt file2 file2.txt
```

### 5. ls - List of files

Description:- The ls command is used to display a list of content of a directory.

Syntax: ls

Command: \$ ls

### 6. Cat

Description:- The command 'cat' is a multi-purpose utility in the linux system. It can be used to create a file, display content of the file, copy the content of one file to another file, and more.

Syntax: cat [OPTION]... [FILE]

To create a file

Syntax:- cat > <file name>  
//Enter file content

→ Press 'CTRL+D' keys to save the file. To display the content of the file, execute it as follows:

Syntax: cat <file-name>

Command:- \$ cat > file1.txt  
//Enter content of the file

\$ cat file1.txt

//It shows file1.txt content

### 7. touch

Description:- It is used to create empty file. We can also create multiple empty files by executing it once.

Syntax:- touch <filename>

touch <file1> <file2> ...

### rm Command

rgukt@rgukt-lenovo-e41-55:~/ankitha \$ rm file1

rgukt@rgukt-lenovo-e41-55:~/ankitha \$ ls

file1.txt file2 file2.txt

### CP Command

file1.txt Contains "Hello RGUKT!". This is copy to new file file2.txt

rgukt@rgukt-lenovo-e41-55:~/ankitha \$ cp file1.txt file2.txt

rgukt@rgukt-lenovo-e41-55:~/ankitha \$ cat file2.txt

Hello RGUKT!

### mv Command

rgukt@rgukt-lenovo-e41-55:~/ankitha \$ mv file2 Desktop

rgukt@rgukt-lenovo-e41-55:~/ankitha \$ ls

~~file1.txt~~

Desktop file1.txt file2.txt

### head Command // file1.txt Contains 1 to 100 numbers

rgukt@rgukt-lenovo-e41-55:~/ankitha \$ head file1.txt

1  
2  
3  
4  
5  
6  
7  
8  
9  
10

Command: \$ touch hello.txt

for creating multiple files

Command: \$ touch file1 file2

### 8. rm - remove

Description: The rm Command is used to remove a file.

Syntax: rm <file name>

Command: \$ rm hello.txt

→ It will remove the file hello.txt.

### 9. cp - Copy

Description - It is used to copy a file or directory

Syntax: To copy in the same directory.

cp <existing filename> <new filename>

Command: \$ cp file1.txt file2.txt

### 10. mv - move

Description: It is used to move a file or a directory from one location to another location.

Syntax: mv <filename> <directory path>

Command: mv file.txt Directory

### 11. head:

Description: It is used to display the content of a file.

It displays the first 10 lines of a file.

### tail Command

// file1.txt contain 1 to 100 numbers

```
rgukt@rgukt-lenovo-e41-55:~ /ankitha $ tail file1.txt
```

```
91
92
93
94
95
96
97
98
99
100
```

### tac Command

// file2.txt contain 1 to 5 numbers

```
rgukt@rgukt-lenovo-e41-55:~ /ankitha $ tac file2.txt
```

```
5
4
3
2
1
```

### more Command

```
rgukt@rgukt-lenovo-e41-55:~ /ankitha $ more hello.txt
```

Hello RGUKT!

Syntax: head <file name>

Command: \$ head file1.txt

### tail

Description:- It is similar to the head Command. The difference between both Commands is that it displays the last ten lines of the file content. It is useful for reading the error message.

Syntax: tail <file name>

Command: \$ tail hello.txt

### 13. tac:-

Description: It is reverse of cat Command, as its name specified. It displays the file content in reverse order (from the last line).

Syntax: tac <file name>

Command: \$ tac hello.txt

### 14. more:-

Description: It is quite similar to the cat Command, as it is used to display the file content in the same way that the cat Command does. The only difference between both Commands is that, in case of larger files, the more Command displays screenful output at a time.

Syntax: more <file name>

Command: \$ more hello.txt

### id Command

```
rgukt@rgukt -lenovo-e41-55:~$ id
uid=1000(rgukt) gid=1000(rgukt) groups=1000(rgukt), 4(adm), 24(cdrom),
27(sudo), 30(dip), 46(plugdev), 120(lpadmin), 131(lxd), 132(sambashare)
```

### clear Command

```
rgukt@rgukt -lenovo-e41-55:~$ clear
```

// It clears the terminal screen, after pressing the ENTER key.

### grep Command

```
// hello.txt contain "Hello RGUKT!"
```

```
rgukt@rgukt -lenovo-e41-55:~$ cat hello.txt | grep Hello
Hello RGUKT!
```

### ping Command

```
rgukt@rgukt -lenovo-e41-55:~$ ping www.google.com
```

```
PING www.google.com (maa03s47-in-x04-1e100.net (2404:6800:4007:82c:
:2004)) 56 data bytes
```

```
64 bytes from maa03s47-in-x04-1e100.net (2404:6800:4007:82c::2004):
```

```
icmp_seq=1 ttl=58 time=614 ms
```

```
1z
```

```
[1]t stopped
```

```
ping www.google.com
```

```
rgukt@rgukt -lenovo-e41-55:~$
```

// After press 'CTRL+Z' it stops.

### 15. id

Description: It is used to display the user ID (UID) and group ID (GID).

Syntax: id

Command: \$ id

// It gives our user ID ~~and group ID~~

### 16. clear

Description: clear Command used to clear the terminal screen.

Syntax: clear

Command: \$ ~~clear~~

// It clears the terminal screen

→ After pressing the ENTER key, it will clear the terminal screen.

### 17. grep

Description: This command is the most powerful and used filter in a Linux system. The 'grep' stands for 'global regular expression print'. It is useful for searching the content from a file. Generally, it is used with the pipe.

Syntax: Command | grep <search word>

Command: \$ ~~grep~~

\$ cat hello.txt | grep hello

### 18. ping

Description: This command is used to check the connectivity between two nodes, that is whether the server is connected. It is a short form of "packet Internet Groper".

Syntax: ping <destination>

Command: \$ ping www.gmail.com

### hostname Command

```
rgukt@rgukt-lenovo-e41-55:~$ hostname  
rgukt-lenovo-e41-55
```

### uniq Command

file1.txt contain 1 to 5 numbers

```
rgukt@rgukt-lenovo-e41-55:~/ankitha $ sort file1.txt | uniq
```

```
1  
2  
3  
4  
5
```

### wc Command

file2.txt contain "Hello RGUKT!"

```
rgukt@rgukt-lenovo-e41-55:~/ankitha $ wc file2.txt
```

```
0 2 12 file2.txt
```

### date Command

```
rgukt@rgukt-lenovo-e41-55:~$ date
```

```
wednesday 06 September 2023 09:50:46 AM IST
```

### 19. hostname

Description: It sets or displays the name of the current host system.

Syntax: hostname

Command: \$ hostname

// It display hostname like

// rgukt-lenovo-e41-55-ankitha-1234

### 20. uniq

Description: It is used to form a sorted list in which every word will occur only once.

Syntax: Command <file name> | uniq

Command: \$ sort marks.txt | uniq

→ It is used to remove the duplicate in the content of the file.

### 21. wc

Description: This command is used to count the lines, words, and characters in a file.

Syntax: wc <file name>

Command: \$ wc hello.txt

### 22. date

Description: It is used to display date, time, time zone and more.

Syntax: date

Command: \$ date

// It prints the present date and time.

// ex: Monday 04 September 2023 09:31:47 PM IST



Cal Command

rgukt@rgukt - lenovo - c41-55: ~\$ cal

September 2023

Su Mo Tu We Th Fr Sa

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

Mount Command

rgukt@rgukt - lenovo - c41-55: ~\$ mountkitha \$ mount

sysfs on /sys type sysfs (rw, nosuid, nodev, noexec, relatime)  
proc on /proc type proc (rw, nosuid, nodev, noexec, relatime, size =  
290163k, nr\_inodes = 792512, mode = 755, (nodev))  
nfs on /run/snapd/ns/snap-store.mount type nfs (rw)

exit Command

rgukt@rgukt - lenovo - c41-55: ~\$ exit

// After ENTER key it exits from terminal.

ip Command

rgukt@rgukt - lenovo - c41-55: ~\$ ip a

1: lo : <loopback, up, lower-up> mtu 65536 qdisc noop state  
UNKNOWN group default qlen 1000

link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00

inet 127.0.0.1/8 scope host lo

valid-lft forever preferred-lft forever

inet6 : ::1/128 scope host

valid-lft forever preferred-lft forever

2: wlp2s0 : <broadcast, MULTICAST> mtu 1500 qdisc noop state  
DOWN group default qlen 1000

link/ether 50:c2:c8:ef:c8:69 brd ff:ff:ff:ff:ff:ff

83. mount

Command : \$ cal

Syntax : cal

- the current date highlighted

Description : It is used to display the current month's calendar with

Description : It is used to connect an external device file system to

the system's file system

Syntax : mount -t type <device> <directory>

Command : \$ mount

85. exit

Description : Linux exit Command is used to exit from the current

shell. It takes a parameter as a number and exits

the shell with a return of status number.

Syntax : -exit

Command : - \$ exit

// It exits from the terminal

86. ip

Description : It is used to as an updated version of the 'pconfig

Command - It is used to assign an IP address,

initialize an interface, disable an interface.

Syntax : ip a (a) ip addr

Command : \$ ip a

## PS Command

rgukt@rgukt -lenovo-e41-55:~\$ ps

| PID  | TTY   | TIME     | CMD  |
|------|-------|----------|------|
| 1801 | pts/0 | 00:00:00 | bash |
| 2293 | pts/0 | 00:00:00 | ping |
| 2501 | pts/0 | 00:00:00 | ps   |

## Kill Command

rgukt@rgukt -lenovo-e41-55:~\$ kill -l

|               |             |             |             |             |
|---------------|-------------|-------------|-------------|-------------|
| 1) SIGHUP     | 2) SIGINT   | 3) SIGQUIT  | 4) SIGILL   | 5) SIGTRAP  |
| 6) SIGABRT    | 7) SIGBUS   | 8) SIGFPE   | 9) SIGKILL  | 10) SIGUSR1 |
| 13) SIGSEGV   | 12) SIGUSR2 | 13) SIGPIPE | 14) SIGALRM | 15) SIGTERM |
| 16) SIGSTKFLT | 17) SIGCHLD | 18) SIGCONT | 19) SIGSTOP | 20) SIGTSTP |
| 21) SIGTTIN   | 22) SIGTTOU | 23) SIGURG  | 24) SIGXCPU | 25) SIGXFSZ |

## Who Command

rgukt@rgukt -lenovo-e41-55:~\$ whoami  
rgukt

## Sleep Command

rgukt@rgukt -lenovo-e41-55:~\$ sleep

// After press ENTER key it goes to sleep mode.

## 27. PS

Description: This command allows you to view all application running and a process number next to it. This is useful to view if applications are running on start-up, or if you have to manual start them.

Syntax: ps

Command: \$ ps

// It display output like

| PID  | TTY   | TIME     | CMD  |
|------|-------|----------|------|
| 3329 | pts/0 | 00:00:00 | bash |
| 3932 | pts/0 | 00:00:00 | ps   |

## 28. kill

Description: This command allows you to terminate a program running from your list of processes. It used to send a signal to running processes.

Syntax: kill -l

Command: \$ kill -l

## 29. who

Description: The who command is used to identifies the users currently logged in

Syntax: whoami

Command: \$ whoami

## 30. Sleep

Description: It is used to hold the terminal by the specified amount of time. By default, it takes time in seconds.

Syntax: sleep <time>

Command: \$ sleep 5

Uthay 25/09/2023



op:

fd=3

Lab-2

- 1 Introduction to System calls - implementation of `open()`, `create()`, `write()`, `read()`, `seek()`
- 2 Implementation of `fork()`, `wait()`, `exec()`, and `exit()` system calls.

open() System call:

Purpose: used to open files and devices for reading and writing

Implementation: when a user process invokes `open()` the operating system kernel checks permissions, resolves the file path, allocates a file descriptor and maintains an internal data structure to track the opened file. It returns a file description to the process.

Syntax: `int open(const char* path, int flag s);`

ex: Program for `open()`

```
#include <errno.h>
```

```
#include <fcntl.h>
```

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
extern int errno;
```

```
int main()
```

```
{
```

```
    int fd=open("foo.txt", O_RDONLY | O_CREAT);
```

```
    printf("fd = %d\n", fd);
```

```
    if (fd == -1) {
```

```
        printf("Error Number %d\n", errno);
```

```
        perror("program");
```

```
    }
```

```
    return 0;
```

```
}
```

O/p:-

fd = 3

file created

## 2) create() system call :-

Purpose:- used to create a new file

Implementation:- Internally, `create()` is similar to `open`. It checks if the file already exists and if not, it creates a new file with the specified name and opens it with write permissions.

Syntax:- `int create (char *file name, mode_t mode);`

```
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
extern int errno;

int main() {
    int fd = create ("file1.txt", O_RDONLY);
    printf ("fd = %d\n", fd);
    printf (fd == -1)
    {
        printf ("Error Number %d\n", errno);
        perror ("program");
    }
    return 0;
}
```

## 3) close() system call :-

Purpose:- used to close an open file

Implementation:- When `close()` is called, the operating system releases any resources associated with the file descriptor, updates file-related data structures, and allows the file descriptor to be reused.

o/p

opened the fd = 3

closed the fd



Syntax: `int close (int fd);`

ex: Program to close()

```
#include <fcntl.h>
```

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main() {
```

```
    int fd1 = open ("foo.txt", O_RDONLY);
```

```
    if (fd1 < 0) {
```

```
        perror ("c1");
```

```
        exit (1);
```

```
    }
```

```
    printf ("opened the fd = %d \n", fd1);
```

```
    if (close (fd1) < 0) {
```

```
        perror ("c1");
```

```
        exit (1);
```

```
    }
```

```
    printf ("closed the fd. \n");
```

```
}
```

4. read() System call:

Purpose: used to read data from an open file

Implementation: When `read()` is called, the OS reads the data from the file, copies it to a user-supplied buffer, and updates the file position. If the end of file is reached, it returns an EOF indicator.

Syntax: `size_t read (int fd, void *buf, size_t cnt);`

O/p:

called read(3, c, 10). returned that 10 bytes were read.  
those bytes are as follows: 0 0 0 foo

Ex: Program to read()

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    int fd, sz;
    char *c = (char *) calloc(100, sizeof(char));
    fd = open("foo.txt", O_RDONLY);
    if (fd < 0) {
        perror("e1");
        exit(1);
    }
    sz = read(fd, c, 10);
    printf("called read(%d, c, 10). returned that \"%d bytes  
were read.\n", fd, sz);

    c[sz] = '\0';
    printf("those bytes are as follows: %s\n", c);
    return 0;
}
```

5) write() system call

Purpose: Used to write data into an open file.

Implementation: When write() is invoked, the OS copies the data from the user space into a kernel buffer, then writes it to the file. It updates the file's metadata and file position.

Syntax: size\_t write(int fd, void \*buf, size\_t cnt);

o/p

called write (3, "hello geeks\n", 12). It returned 11

Ex: program to write()

```
#include <stdio.h>
```

```
#include <fcntl.h>
```

```
main()
```

```
{  
    int sz;
```

```
    int fd = open("foo.txt", O_WRONLY | O_CREAT | O_TRUNC,  
0644);
```

```
    if (fd < 0)
```

```
    {  
        perror("r");
```

```
        exit(1);
```

```
    }  
    sz = write(fd, "hello geeks\n", strlen("hello geeks\n"));
```

```
    printf("called write (%d, \"hello geeks\n\n\", %d)."
```

```
"It returned %d\n", fd, strlen("hello geeks\n"), sz);
```

```
    close(fd);
```

```
}
```

6) seek() System call :-

Purpose :- used to change the position of the file pointer within an open file.

Implementation: seek() allows you to move the file pointer to a specified offset from the beginning current position, or end of the file. Its often used for random access in files.

Syntax: off\_t seek (int fd, off\_t offset, int whence);

Q/P

abcde \* 12345 \* fghij \* 67890

Ex: program to lseek()

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int n, f, fi;
    char buf[6];
    f = open("seek", OR O_RDWR);
    read(f, buf, 6);
    write(1, buf, 6);
    lseek(f, 5, SEEK_CUR);
    read(f, buf, 6);
    write(1, buf, 6);
}
```

2) 1) fork():

Purpose: fork() is used to create a new process (child process) that is a copy of the calling process (parent process). The child process is almost identical to the parent but has its own unique process ID (PID).

Implementation: When fork() is called, the operating system creates a new process, duplicating the memory space, file descriptors and other attributes of the process. The child process starts executing from the same point in the code where the fork() call was made but has its own stack and data segments.



o/p

Hello world!

Hello world!

o/p

HC: hello from child

Bye

HP: hello from parent

CT: child has terminated.

Bye

ex: program of fork()

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
    fork();
    printf("Hello world");
    return 0;
}
```

2) wait():

Purpose: wait() is used by a parent process to wait for its child process to terminate. It allows the parent to obtain the exit status of the child process.

Implementation: When a parent process calls wait(), it is suspended until one of its child processes exits. The OS keeps track of the child processes and their exit statuses.

Syntax: pid\_t wait(int \*stat\_loc);

ex: program to wait()

```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

int main()
{
    if (fork() == 0)
        printf("HC: hello from child\n");
    else
        printf("HP: hello from parent\n");
    wait(NULL);
    printf("CT: child has terminated\n");
}
```

O/p

Parent pid = 12345678

child pid = 89546848

```
printf("Bye\n");  
return 0;  
}
```

(B)

```
#include <stdio.h>  
#include <stdlib.h>  
#include <sys/wait.h>  
#include <unistd.h>  
  
int main() {  
    pid_t cpid;  
    if (fork() == 0)  
        exit(0);  
    else  
        cpid = wait(NULL);  
    printf("parent pid = %d\n", getpid());  
    printf("child pid = %d\n", cpid);  
    return 0;  
}
```

Exit():-

The exit() is such a function (or) one of the system calls that is used to terminate the purpose process. This system call defines that the execution is completed especially in the case of a multi-threaded environment. For future reference, the status of the process is captured.

After the use of exit() system call, all the resources of the use of exit() process are retrieved by the operating system & then terminate the process. The system call exit() is equivalent to \_exit().

Syntax: void exit(int status);

## Output

Error opening file

ex1' Program to exit()

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE * PFile;
    PFile = fopen("myfile.txt", "r");
    if (PFile == NULL)
        printf("Error opening file");
        exit(1);
    }
    else {
        }
    return 0;
}
```

Exec() ← The exec() is such a system call that runs by replacing the current process image with the process image. However, the original process remains as a new process but the new process ~~remains a~~ replaces the heap data, stack data etc.. It runs the program from the entry point by loading the program into the current process space.

Syntax: int execv(const char \*path, char \*const argv[]);

## Program

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    int i;
    printf("I am exec-c called by execv()");
}
```

Chhaya 21/10/2023

Aim: Write a program to simulate the following preemptive CPU scheduling algorithms.

a) FCFS (First Come First Serve)

Program:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    n = printf("Enter no. of processes:");
```

```
    scanf("%d", &n);
```

```
    int AT[n], BT[n], temp, i, j, temp1, temp2, pid[n];
```

```
    printf("Enter the pid of the processes:");
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &AT[i]);
```

```
    }
```

```
    printf("Enter the burst time of the processes");
```

```
    for (i = 0; i < n; i++) {
```

```
        scanf("%d", &BT[i]);
```

```
    }
```

```
    // Sorting the process according to arrival time.
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        for (j = i + 1; j < n; j++)
```

```
        // Sorting the process having less arrival time
```

```
        if (AT[i] > AT[j])
```

```
        {
```

```
            temp = AT[i];
```

```
            AT[i] = AT[j];
```

```
            AT[j] = temp;
```

```
            temp1 = BT[i];
```

```
            BT[i] = BT[j];
```

```
            BT[j] = temp1;
```

## output

Enter no. of processes : 5

Enter no. of pid of processes : 2 4 1 0 3

Enter the arriving time of the processes : 0 0 1 1 2

Enter the burst time of the processes : 2 3 4 5 6

| Pid | Arrival time | Burst time | waiting time | Turnaround time |
|-----|--------------|------------|--------------|-----------------|
| 2   | 0            | 2          | 0            | 2               |
| 4   | 0            | 3          | 2            | 5               |
| 0   | 1            | 5          | 4            | 9               |
| 1   | 1            | 4          | 9            | 13              |
| 3   | 2            | 6          | 12           | 16              |

average waiting time is 5.000000

average turn around time is 9.000000

```
temp2 = pid[i];
```

```
pid[i] = pid[i+1];
```

```
pid[i+1] = temp2;
```

```
}
```

// if two process have same arrival time then sort them having less process id

```
else if (AT[i] == AT[i+1]) {
```

```
    if (pid[i] > pid[i+1]) {
```

```
        {
```

```
            temp = AT[i];
```

```
            AT[i] = AT[i+1];
```

```
            AT[i+1] = temp;
```

```
            temp1 = BT[i];
```

```
            BT[i] = BT[i+1];
```

```
            BT[i+1] = temp1;
```

```
            temp2 = pid[i];
```

```
            pid[i] = pid[i+1];
```

```
            pid[i+1] = temp2;
```

```
}
```

```
}
```

```
}
```

```
}
```

```
int c[n], w[n], total_wt=0, total_tat=0;
```

```
w[0] = 0;
```

```
c[0] = AT[0] + BT[0];
```

```
for (i=1; i<n; i++)
```

```
{ if (c[i-1] > AT[i])
```

```
{ w[i] = c[i-1] - AT[i];
```

```
c[i] = c[i-1] + BT[i];
```

```
}
```

```
else {
```

```
    w[i] = 0;
```

```

        C[i] = AT[i] + BT[i];
    }

    }

    for (i=0; i<n; i++){
        total_tat += (w[i] + BT[i]);
        total_wt += w[i];
    }

    float avg_wt, avg_tat;
    avg_wt = total_wt/n;
    avg_tat = total_tat/n;

    printf("pid");
    printf("\t\t Arrival time");
    printf("\t\t Burst time");
    printf("\t\t waiting time");
    printf("\t\t turnaroundtime");
    printf("\n");
    for (i=0; i<n; i++){
        printf("%d\t\t\t\t %d\t\t\t\t %d\t\t\t\t %d\t\t\t\t %d\t\t\t\t", P[i], AT[i], BT[i], w[i],
            (w[i] + BT[i]));

        printf("\n");
    }

    printf("average waiting time is %f\n", avg_wt);
    printf("Average turnaroundtime is %f", avg_tat);
    return 0;
}

```

### Output

Enter the no of processes : 5

Enter the process id 4 3 0 1 2

Enter the Arrival time 0 0 1 1 0

Enter the burst time 2 2 3 5 6

| pid | Arrival time | Burst-time | waiting time | Turnaround time |
|-----|--------------|------------|--------------|-----------------|
| 3   | 0            | 2          | 0            | 2               |
| 4   | 0            | 2          | 2            | 4               |
| 0   | 1            | 3          | 6            | 11              |
| 1   | 1            | 5          | 6            | 18              |
| 2   | 0            | 6          | 12           |                 |

Average waiting time is 4.000000

Average Turn around time is 8.000000

b) SJF (Shortest job First) Non preemption.

### Algorithm

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter the number of processes:");
```

```
    scanf("%d", &n);
```

```
    int AT[n], BT[n], wt[n], CT[n], pos, pid[n], i, j, min=1000,
```

```
    temp, temp1, temp2, total_tat, total_wt, flag;
```

```
    printf("Enter the process ids");
```

```
    for(i=0; i<n; i++) {
```

```
        scanf("%d", &pid[i]);
```

```
    }
```

```
    printf("Enter the arrival time");
```

```
    for(i=0; i<n; i++) {
```

```
        scanf("%d", &AT[i]);
```

```
    }
```

```
    printf("Enter the burst time");
```

```
    for(i=0; i<n; i++) {
```

```
        scanf("%d", &BT[i]);
```

```
    }
```

```
    //Sorting the processes according arrival and burst time
```

```
    for(i=0; i<n; i++) {
```

```
        for(j=i+1; j<n; j++)
```

```
        {
```

```
            //Sort the processes having less arrival time
```

```
            if(AT[i] > AT[j])
```

```
            {
```

```
                temp = AT[i];
```

```
                AT[i] = AT[j];
```

```
                AT[j] = temp;
```

```
                temp1 = BT[i];
```

```
                BT[i] = BT[j];
```

```
                BT[j] = temp1;
```



temp2 = pid[i];

pid[i] = pid[j];

pid[j] = temp2;

}

// if two processes have same arrival time, then sort them.

// having less burst time

else if (AT[i] > BT[j])

{  
  if (BT[i] > BT[j])

  {  
    temp = AT[i];

    AT[i] = AT[j];

    AT[j] = temp;

    temp1 = BT[i];

    BT[i] = BT[j];

    BT[j] = temp1;

    temp2 = pid[i];

    pid[i] = pid[j];

    pid[j] = temp2;

  }

else if (BT[i] == BT[j])

{  
  if (pid[i] > pid[j])

  {  
    temp = AT[i];

    AT[i] = AT[j];

    AT[j] = temp;

    temp1 = BT[i];

    BT[i] = BT[j];

    BT[j] = temp1;

    temp2 = pid[i];

    pid[i] = pid[j];

    pid[j] = temp2;

}  
}  
}

```

w[0] = 0
C[0] = AT[0] + BT[0]
for (i=1; i<n; i++)
{
    for (j=i; j<n; j++)
    {
        if (AT[i] <= C[i-1])
        {
            if (BT[j] < min)
            {
                min = BT[j];
                pos = j;
            }
            else if (BT[j] == min)
                break;
        }
        flag = 0;
    }
    else
        flag = 1;
}

if (flag == 0) {
    temp = AT[i];
    AT[i] = AT[pos];
    AT[pos] = temp;
    temp1 = BT[i];
    BT[i] = BT[pos];
    BT[pos] = temp1;
    temp2 = pid[i];
    pid[i] = pid[pos];
    pid[pos] = temp2;
    min = 1000;
    C[i] = C[i-1] + BT[i];
    w[i] = C[i-1] * -AT[i];
}

else if (flag == 1)
{
    w[i] = 0;
}

```

[illegible]